

香港中文大學
The Chinese University of Hong Kong

Agent-Guided Cross-Layer Chiplet Optimization

Academic Challenge 2: Automated Cross-Layer Optimization
of Integrated Chips Using Agentic AI

FlynnCarry Team

Shuo Ren[1] Libo Shen[1] Rongliang Fu[1] Tsung-Yi Ho[1], Fellow, IEEE

[1] The Chinese University of Hong Kong

9 October 2026 · Shanghai

Outline

01 Challenge and Workloads 3–5

Objectives, physical constraints, MLP and Transformer.

02 Agent-Guided Chiplet DSE 6–14

Search and evaluation, followed by three configuration comparisons.

03 Workload Execution Optimization 15–21

Diagnose costs, then reduce CPU–GPU transfers through sharding and fusion.

04 Results and Conclusion 22–23

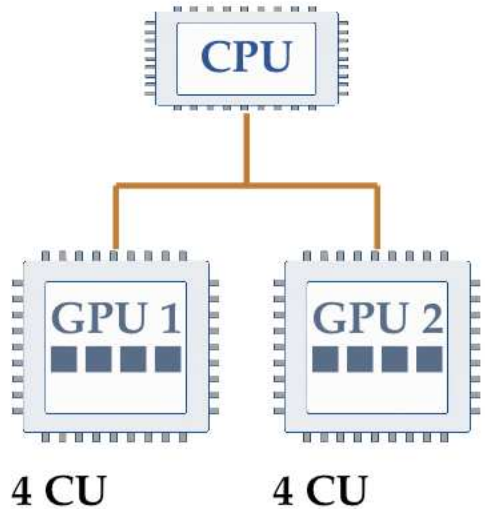
ROI, full-process and end-to-end gains for both workloads.

Competition Requirements

Workloads

MLP training
Transformer forward

Candidate system



Objective

Runtime

Minimize cycles
per workload weight

Stage 1: evaluation

Build the evaluation flow
Measure runtime and physical feasibility

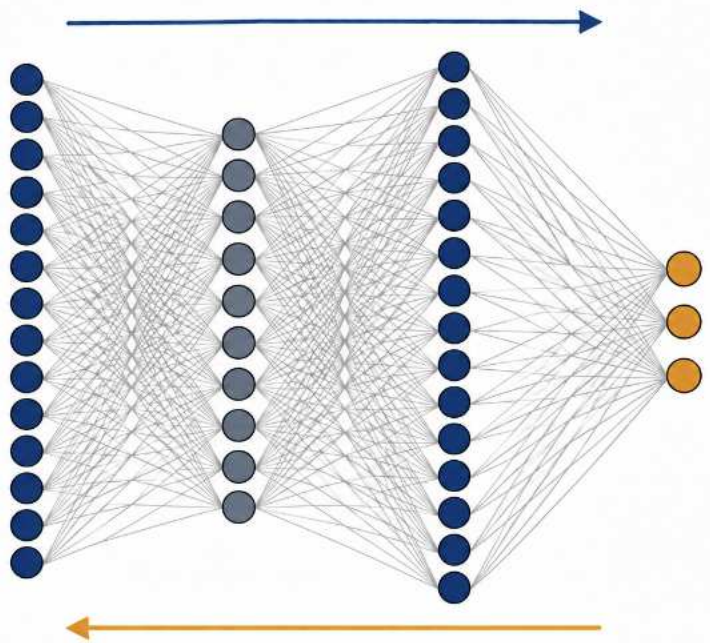
Stage 2: Agent search

Search hardware and mapping choices
Select a feasible design for each workload

≤8 chiplets ≤800 mm² ≤700 W ≤80°C

Workload 1: MLP: One Batch, One Weight Update

Input layer: 13 features per sample
Hidden layers: 10 and 15 neurons
Output layer: 3 values per sample



01 Forward

142 samples, each with 13 input features.

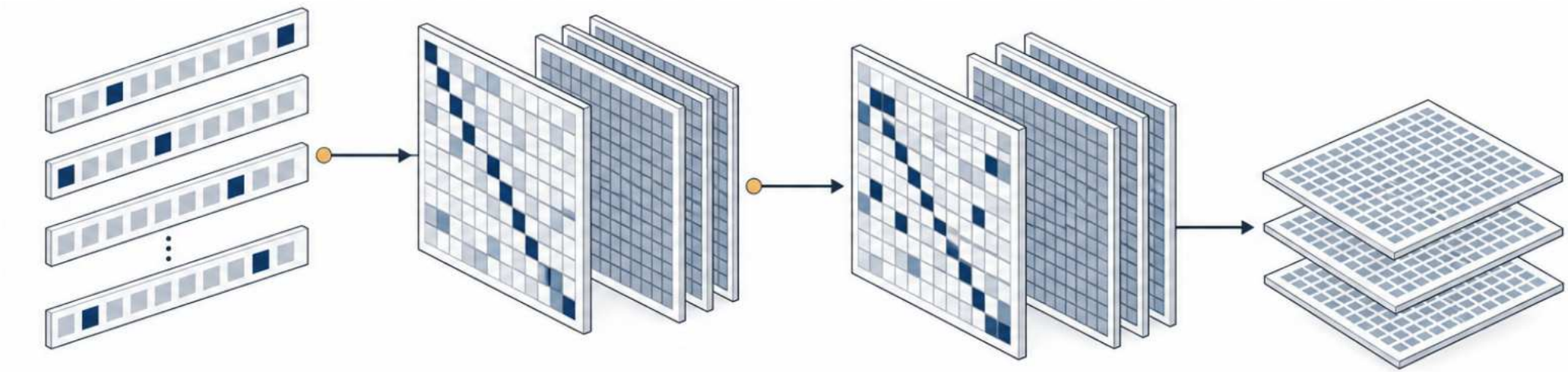
02 Backward

Compute prediction errors and weight gradients.

03 One update

Merge gradients, apply global clipping, then update weights.

Workload 2: Transformer: Ten Complete Sequence Computations



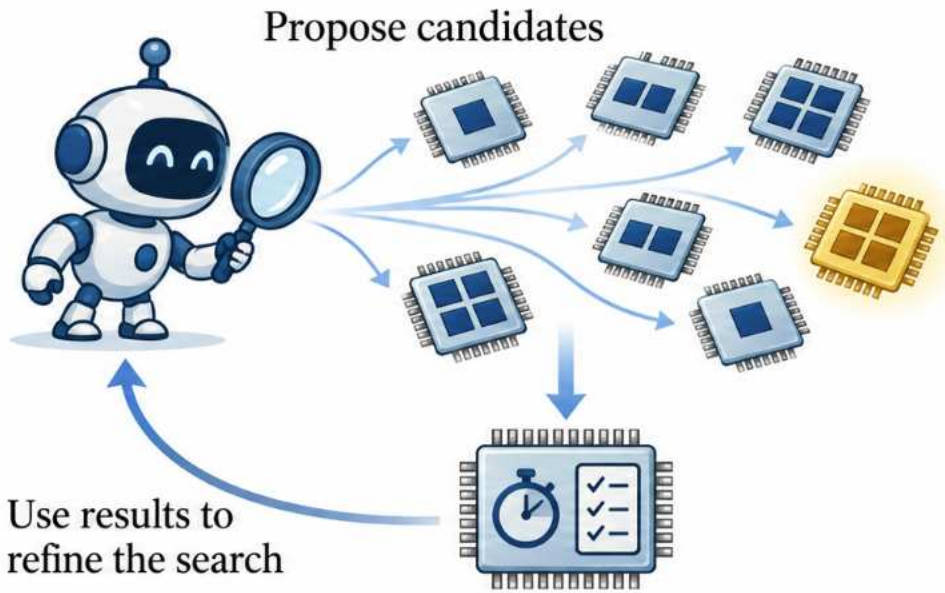
Input sequences **Encoder** **Decoder** **Full logits**

10 sequences × 100 tokens
512 dimensions, 8 attention heads

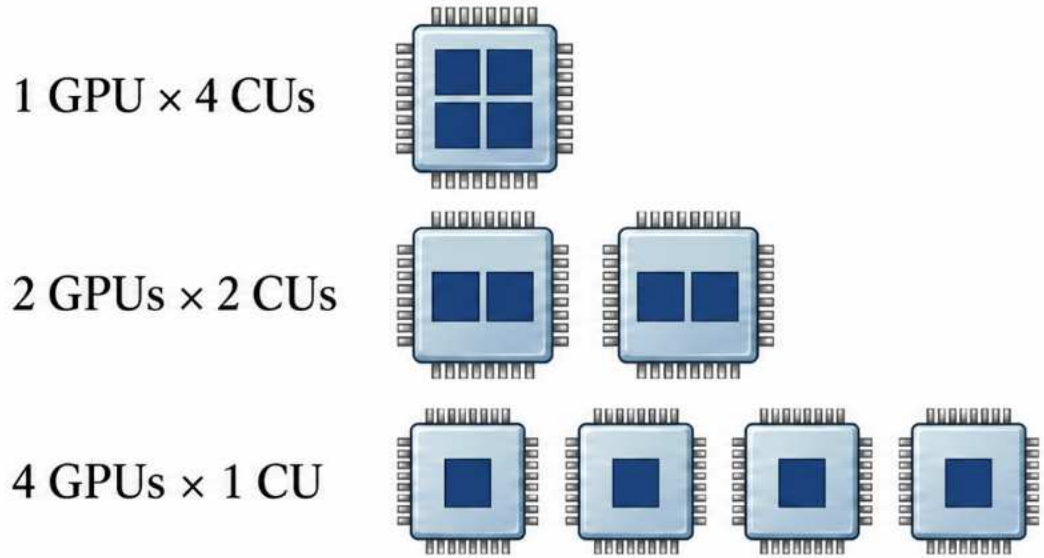
Attention + FFN in both stages
Output: 10 × 100 × 1024 logits

Hardware Design Space

Agent-guided search



Same total CUs, different distributions



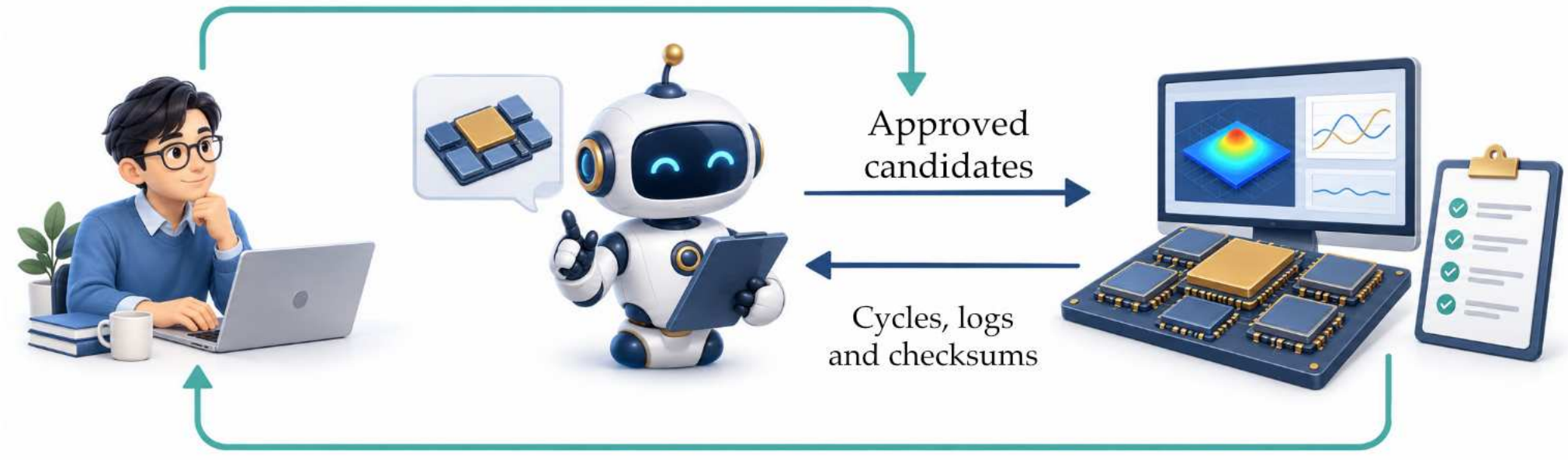
Each example has 4 total CUs.

CPU: 1–4 chiplets, 1–8 cores each
GPU: 1–4 chiplets, 1–32 CUs each

Our coordinator:
one CPU chiplet

Agents Search; Humans Govern the Experiment

Set goals and constraints. Authorize changes.



Review evidence and refine the optimization hypothesis

Human

Set goals and constraints.
Review the evidence.
Approve changes.

Planner Agent

Propose candidate configurations.
Use measured results to
refine the next proposal.

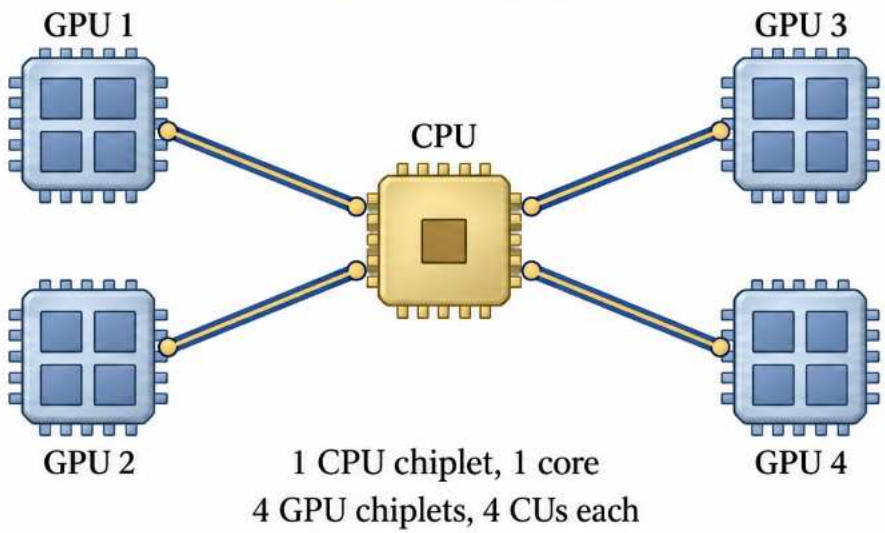
Evaluation environment

Run the benchmark in LEGO-Sim.
Check physical constraints.
Record logs and checksums.

A Concrete Design Point

One MLP candidate

Hardware



Network

Star topology

Flit size: 128

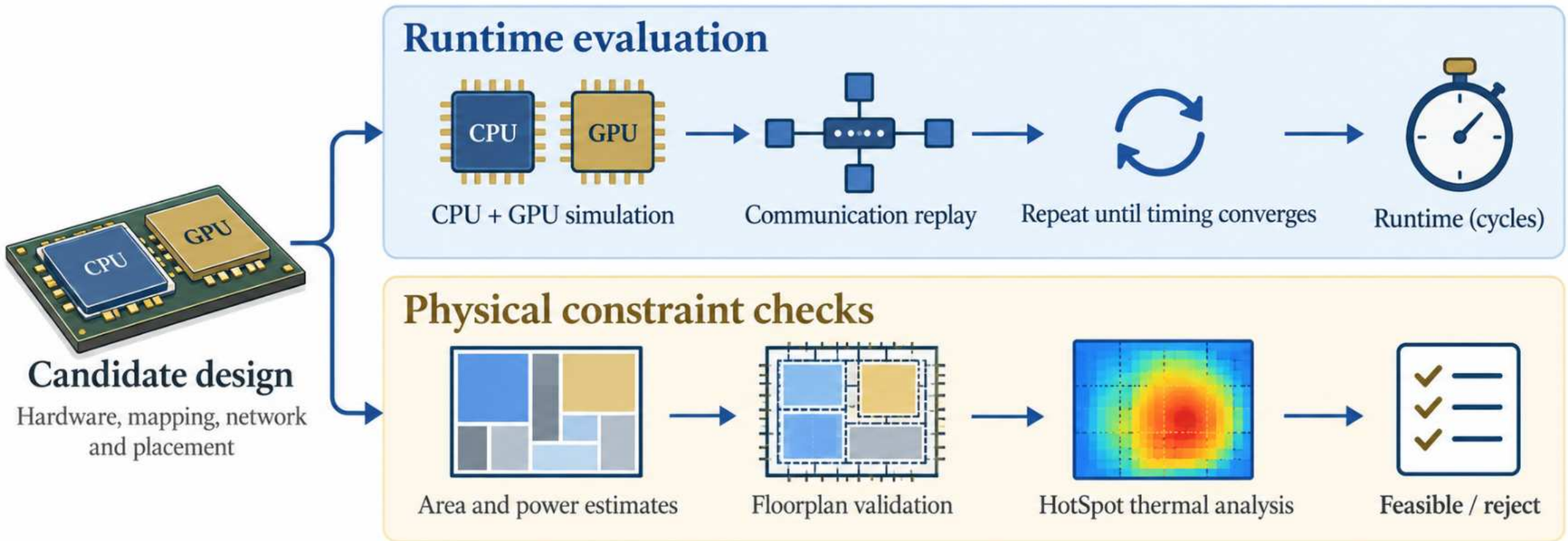
Virtual channels: 6

Buffer setting: 24

LegoSim already provides chiplet interconnect simulation and multiple topology options.
We include topology, flit size, virtual channels and buffers in the design-point search.
We evaluate the same workload across network settings. The diagram shows one candidate.

For this search, we fix 1 CPU and 4 GPUs and vary cores, CUs and network settings.
We check runtime and physical constraints.

Every Candidate Passes Through the Same Evaluator



We compare the runtime of candidates that satisfy all physical limits.

Configuration Search: Results for Both Workloads

Score = Runtime / workload weight; lower is better.
MLP weight: 1.0 Decoder weight: 1.5

Workload	Initial latency (cycles)	After search (cycles)	Speedup	Initial Score	After search Score
MLP training	161.23M	47.43M	**3.40×**	161.23M	47.43M
Transformer decoder	1,073.06M	813.09M	**1.32×**	715.37M	542.06M

What have we achieved at this stage?

The MLP improves by 3.40×, while the decoder improves by 1.32×.
These gains come from hardware and network configuration search.
We use an initial compliant configuration as the decoder baseline.

Understanding the Configuration Search Gains



The search has already improved latency for both workloads.
We now compare parameters to explain why the selected configurations are faster and which changes provide little additional benefit.

1. Interconnect structure

Compare topologies to explain the benefit of the selected interconnect.

2. Communication parameters

Compare flit widths, buffers and virtual channels to identify diminishing gains.

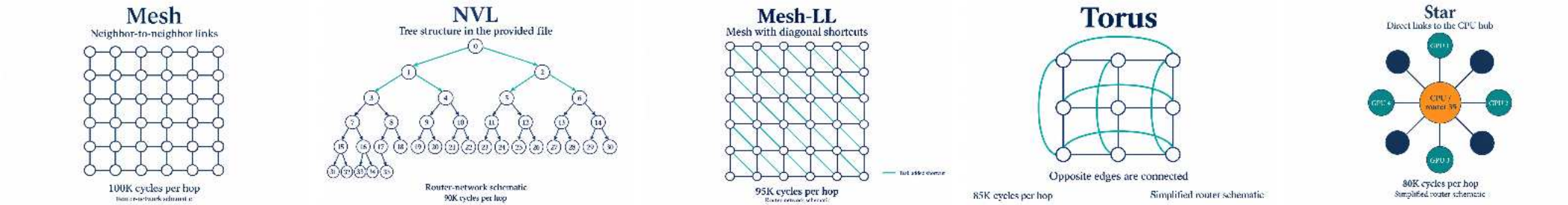
3. Compute resources

Compare CPU cores and GPU CUs to see whether extra resources reduce latency.

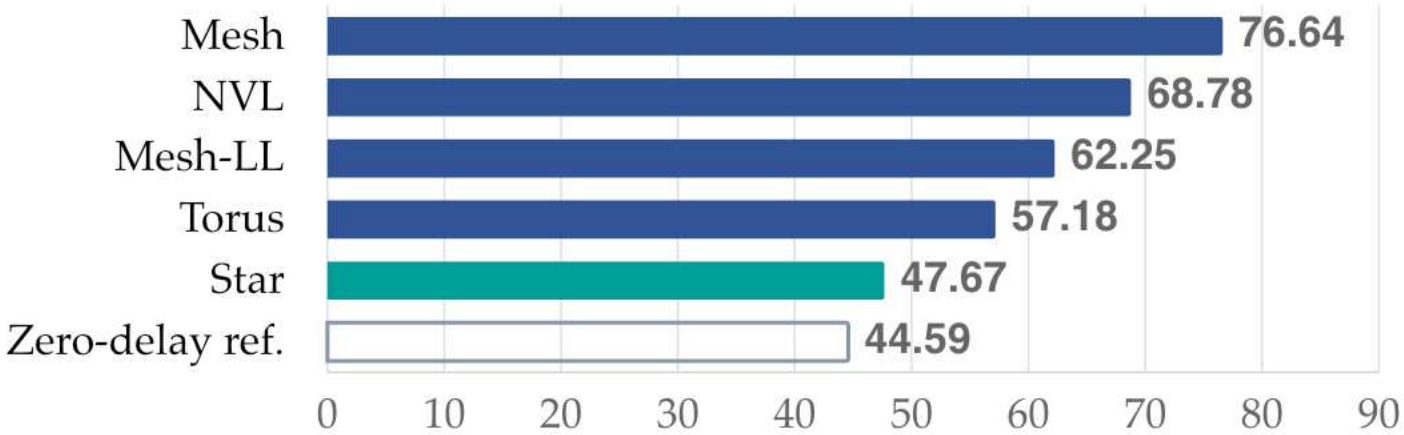
The tested configuration gains diminish. We next examine workload execution costs.

1. Interconnect: Gains and Remaining Headroom

Stock MLP, v8; 1 CPU core, 4 GPUs × 4 CUs; flit 128, VC 6, buffer 24.



Latency (M cycles)



1.61× faster

Mesh to Star: 37.8% lower latency.

Star: 1 hop × 80K
Mesh: 7–9 hops × 100K

6.5% gap to zero-delay first round.

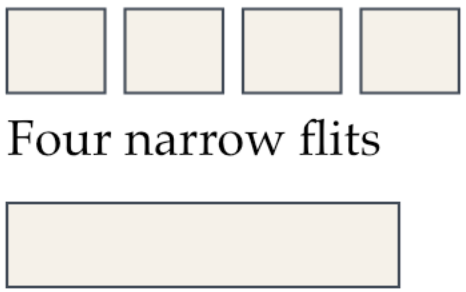
Finding: further network tuning has limited headroom for this stock MLP.

Nearby Star runs: 47.32–47.69M. Network files change both connections and per-hop delay.
Zero-delay first round is an ideal reference, not a proven global lower bound.

2. Communication Parameters: Transfer Width

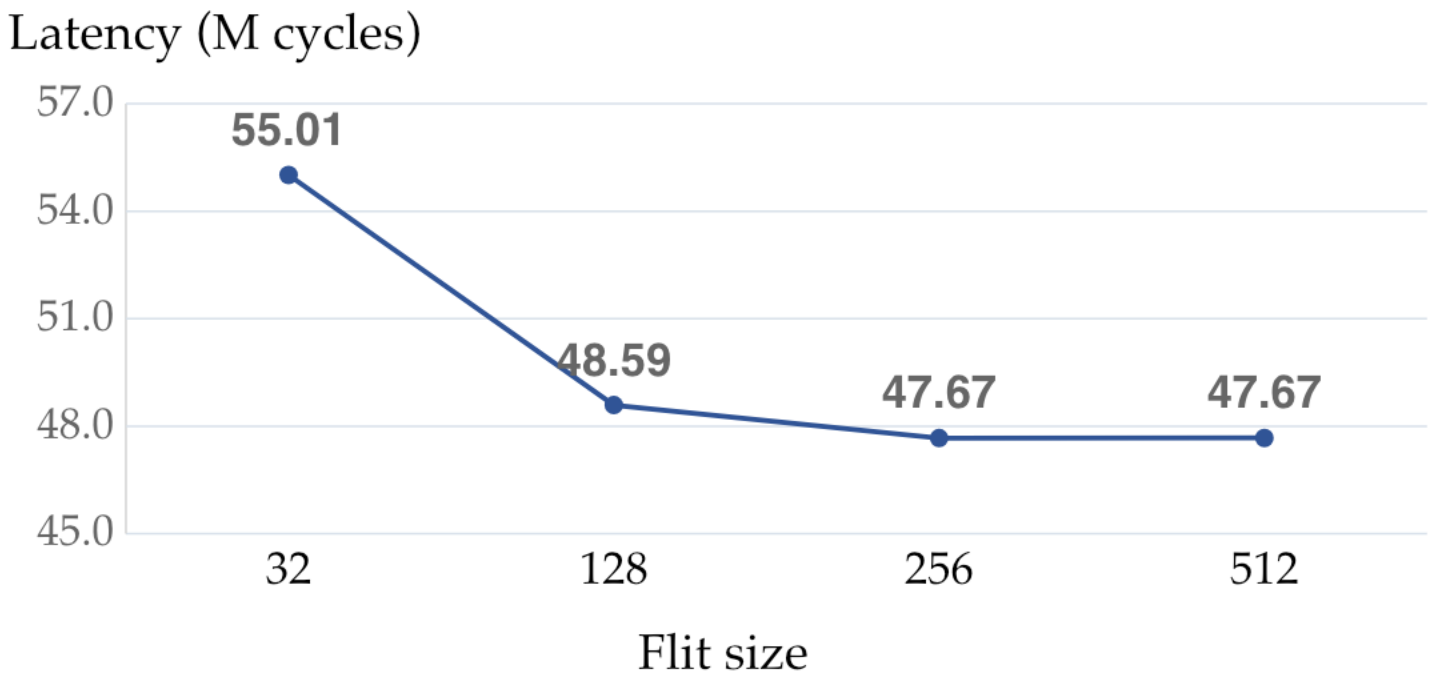
We next test whether wider transfers continue to reduce MLP latency.

Same payload (illustration)



Four narrow flits

One wider flit

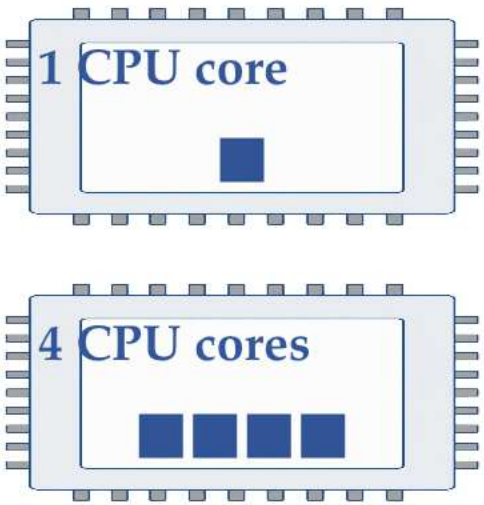


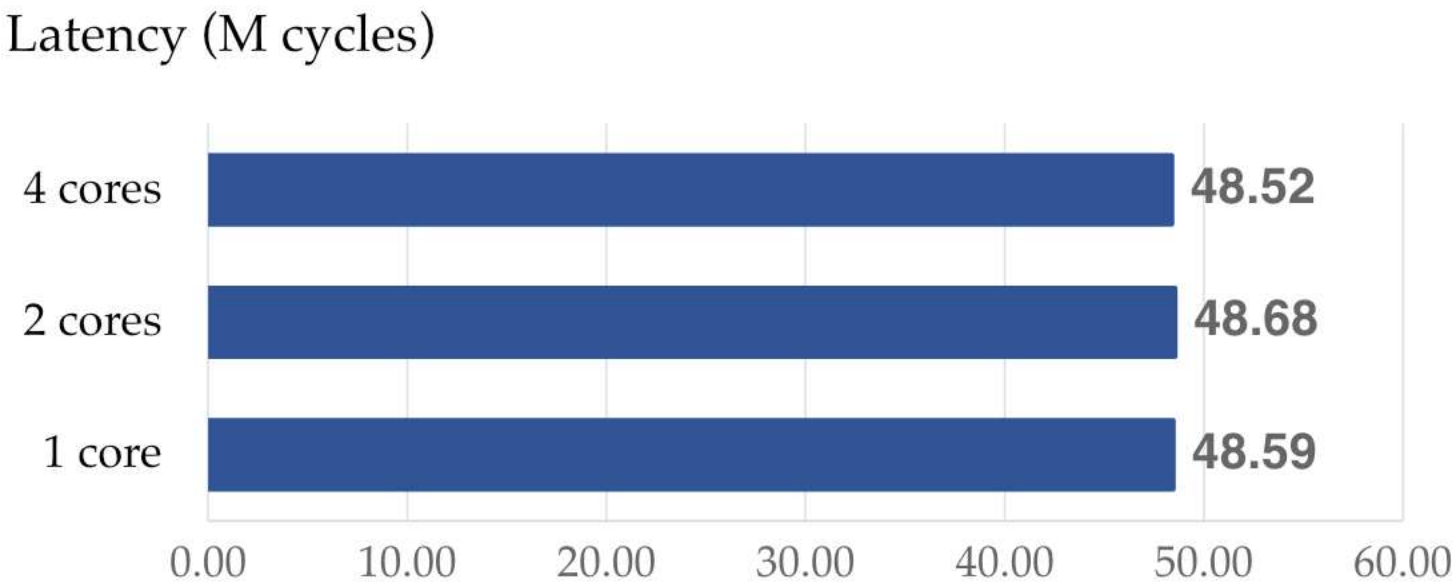
Finding: 256 and 512 both take about 47.67M cycles.

Doubling the flit size again gives almost no additional benefit.
Buffer 12 → 24 gives about 2.3% in a separate matched comparison;
more virtual channels show little benefit in the tested stock MLP.

3. Compute Resources: CPU Cores and GPU CUs

We then ask whether more compute resources can reduce the remaining latency.





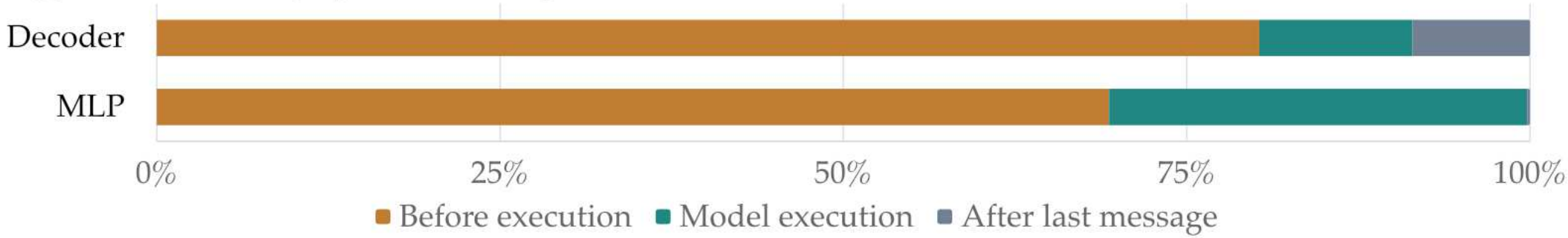
Finding: 1, 2 and 4 cores all take about 48.6M cycles for MLP.

Extra GPU CUs also give little benefit for the stock MLP in our tests.
These comparisons show diminishing gains within the tested configurations.
We next change workload execution, then adapt resources to its new parallelism.

Diagnosis: Preparation and Model Execution

Yellow: preparation. Green: model execution, including CPU work and transfers.

Approximate latency split after configuration search (%)



MLP breakdown: 47.43M cycles in total

Task / time window	Who executes	M cycles
Runtime startup	CPU / runtime	2.32
MNSIM exchange	CPU + MNSIM + links	0.16
JSON + debug output	CPU	30.45
Model execution window	CPU + GPU + links	14.38
After last message	CPU completion / tail	0.12

Green includes three executors

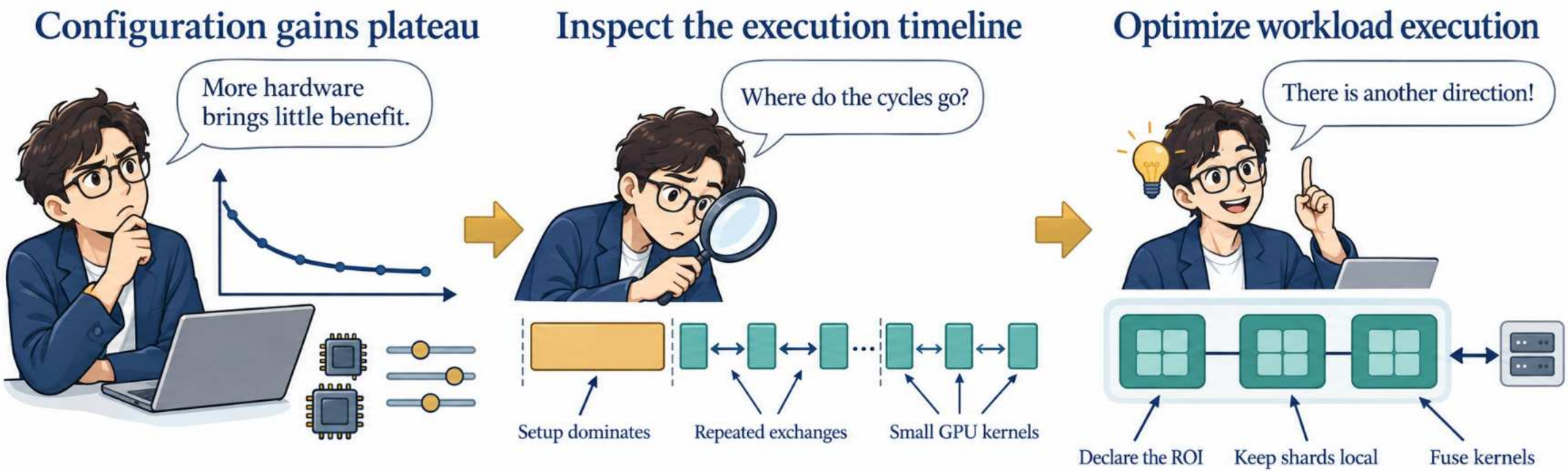
CPU: dispatch, assemble results, activation and other host work.
GPU: matrix computation.
Links: input and result transfers.

Decoder yellow (~80%): CPU startup, construction and embedding.

Cleanup / -O2 target CPU preparation. Sharding / fusion target execution and transfers.

Message timestamps define these windows; CPU/GPU time is not independently profiled.

Why We Optimize the Workload Next



For the stock MLP, tested configuration changes provide diminishing gains.
We reduce preparation overhead through program cleanup and compiler optimization.
We then improve training through sample sharding and kernel fusion.
As execution changes, we also adapt GPU CUs, placement and the link model.

Why We Turn to Workload Optimization



1. Configuration tuning offers limited additional gains

Topology, transfer width and compute resources have been tested.
Further tuning gives small gains for the stock MLP.

2. The remaining costs come from the program

Preparation: remove debug output and use -O2 compilation.
Execution: reduce repeated CPU–GPU transfers and small-kernel overhead.

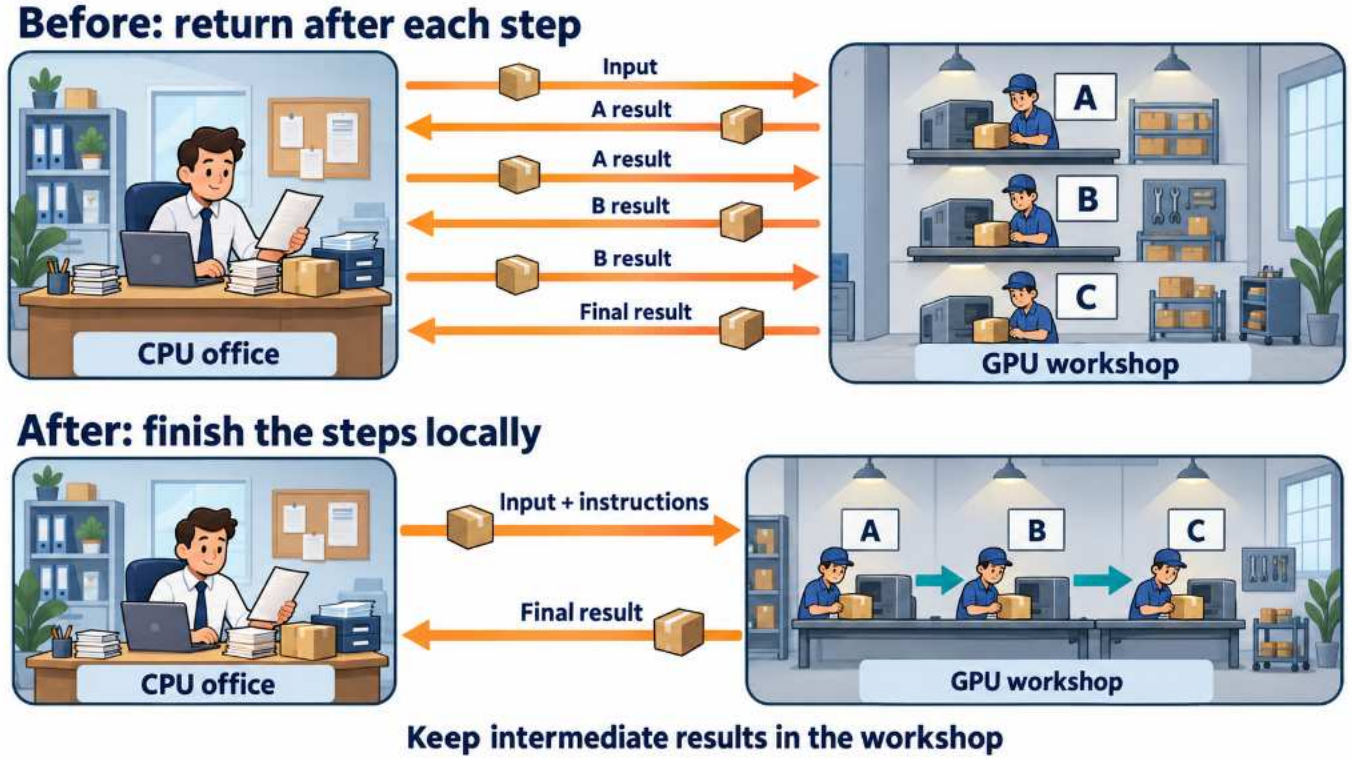
3. Focus next on model execution inside the ROI

Measure input transfers, GPU computation, returned results
and CPU merge, clipping and update within the execution window.

Next: restructure the workload so GPUs can complete more work locally.

ROI defines the measurement scope. We also report full-process latency.

Overall Effect: Fewer CPU–GPU Round Trips



CPU: dispatch a shard once

Send data and run parameters.
GPU code is already compiled.

GPU: keep data local

Reuse the shard and intermediate activations in shared memory.

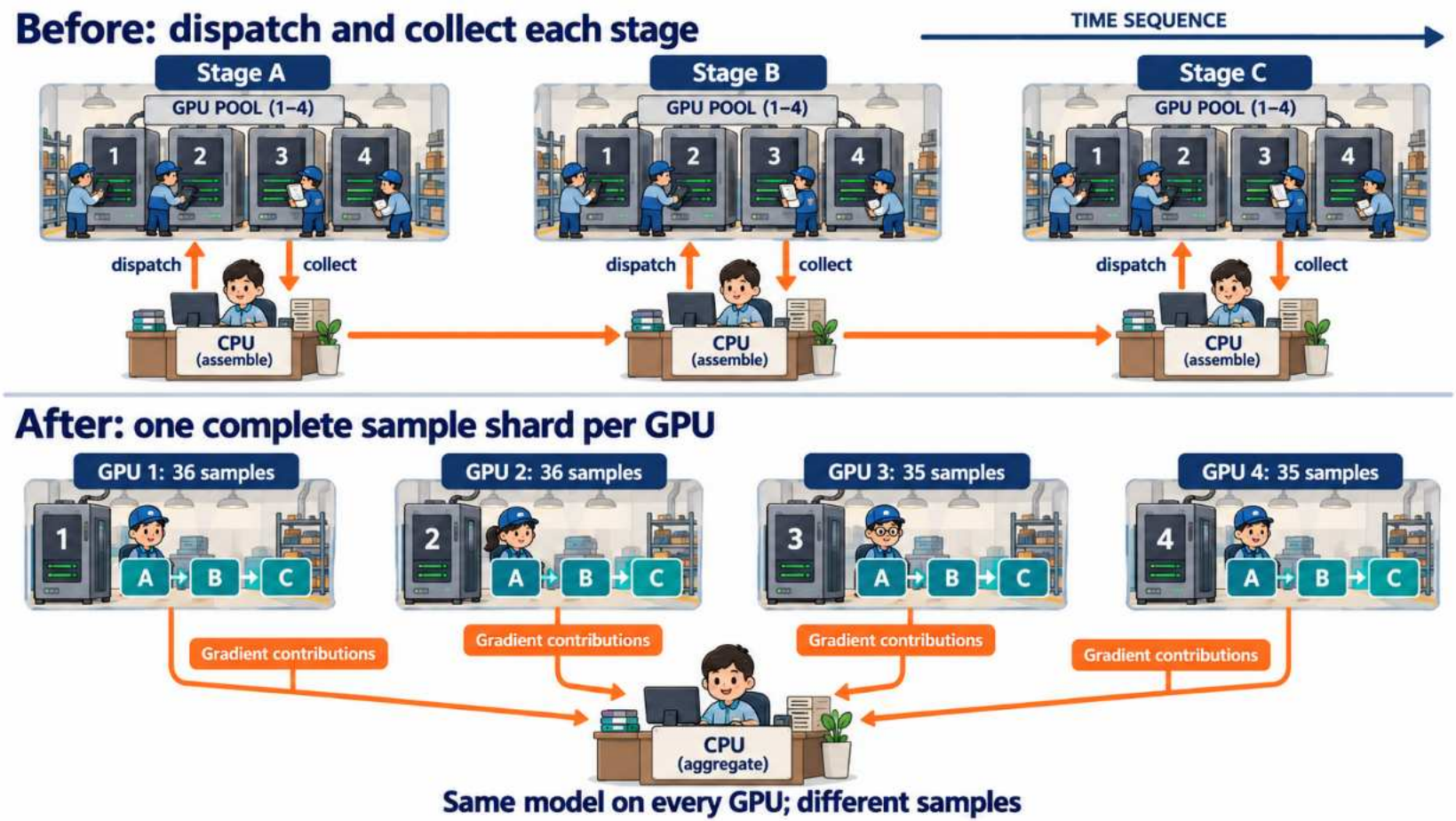
GPU: execute in order

One fused kernel runs all 8 stages.
Block barriers preserve order.

Two changes achieve this effect: sample sharding (P19) + kernel fusion (P20).

This small MLP fits in one thread block; larger working sets need another execution plan.

How Do Four GPUs Divide the Work?



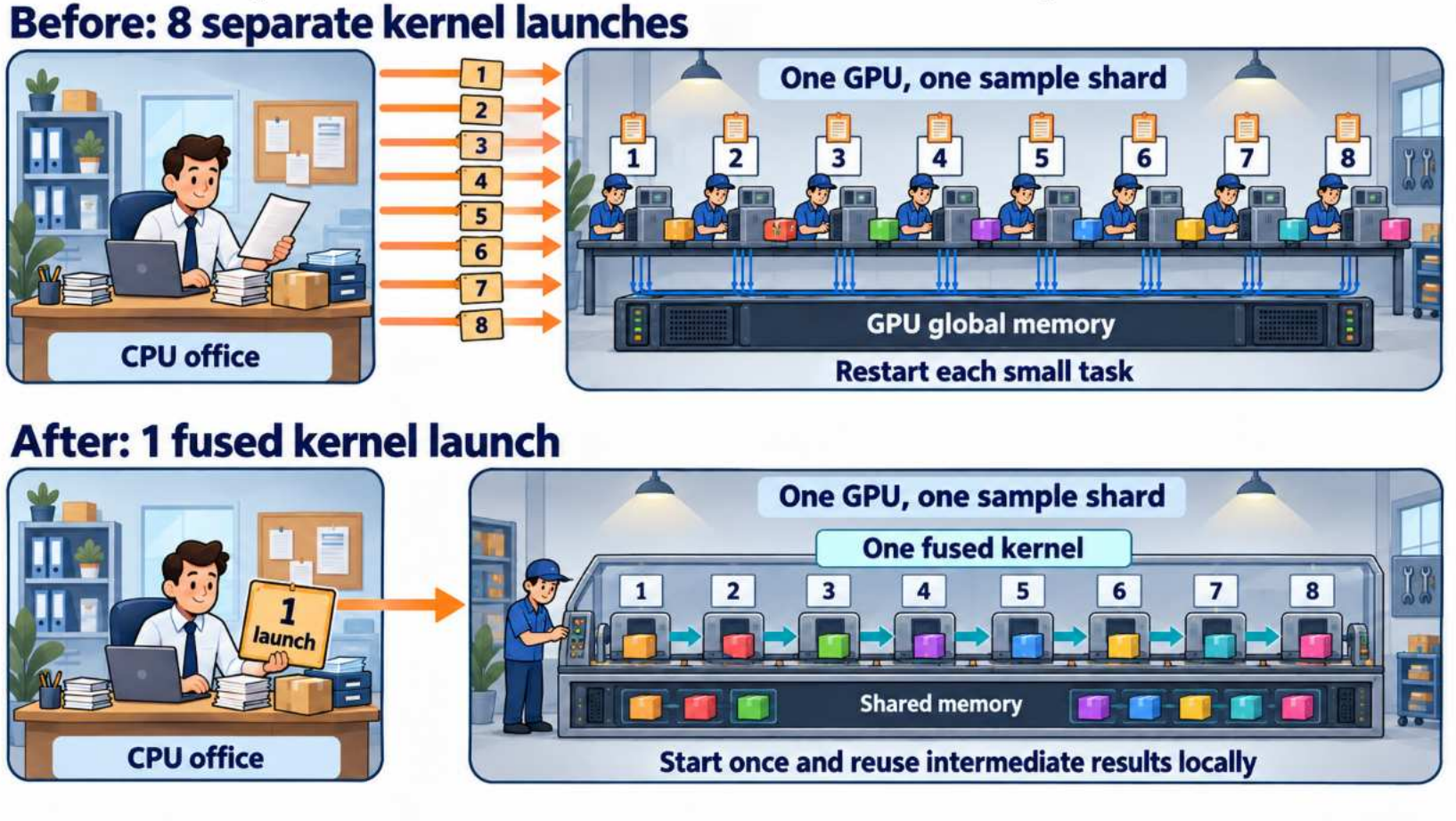
Before: operator-level matrix partitioning, with CPU round trips between stages.

Method 1: sample sharding. Each GPU runs the complete model on different samples.

Together with kernel fusion (P20), this achieves fewer CPU–GPU round trips (P18).

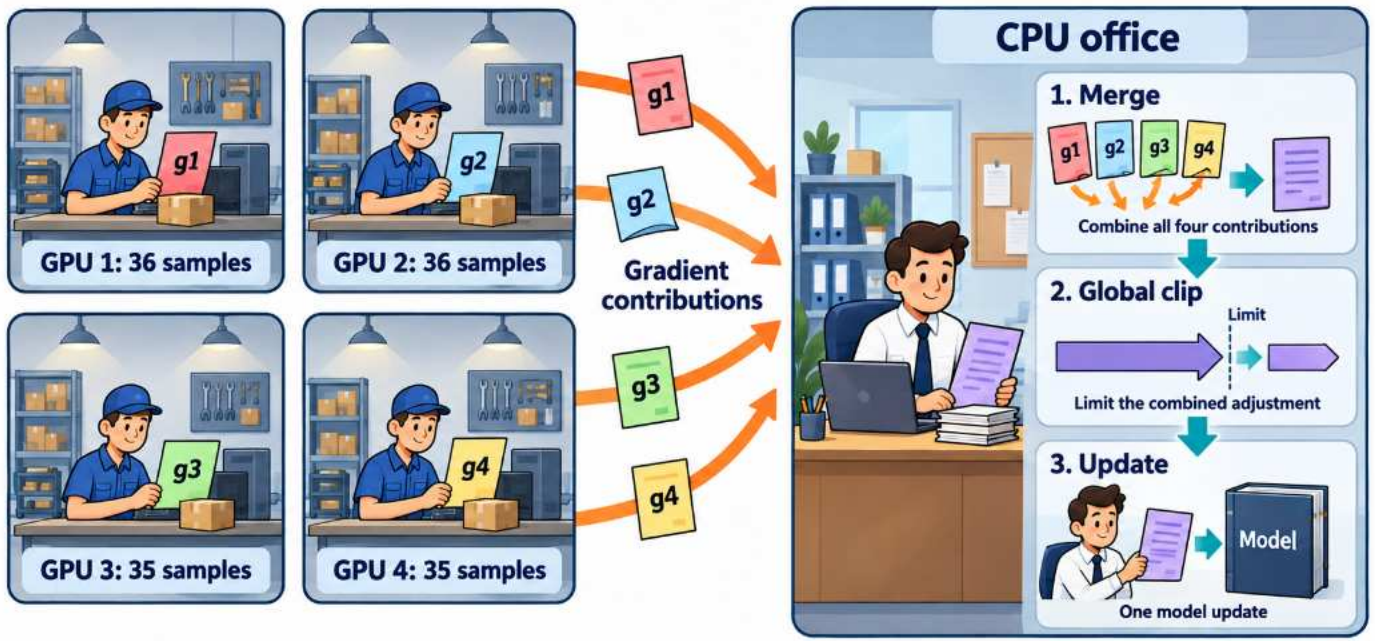
How Does Each GPU Execute Its Work?

Take GPU 1 as an example: it has received its shard of 36 samples.



Method 2: kernel fusion. Run each sample shard in one kernel to achieve the P18 effect. One thread block reuses shared memory across all eight stages.

Four GPU Contributions, One Model Update



Four shards contribute to one full-batch update

Current: reduce to the CPU

CPU merges all contributions, then applies full-batch clipping and updates the model.

All-reduce: every GPU receives

All participants receive the same reduced result and can keep their model replicas consistent.

If another step follows

Before the next pass, broadcast updated weights to all GPUs.

The current benchmark ends after one full-batch update, so no weight broadcast is needed.

For repeated training: CPU reduce + update + broadcast keeps all model replicas consistent.

Results Summary

1. Chiplet DSE: stock program, full-process latency (M cycles)

Workload	Initial	After DSE	Speedup
MLP	161.23	47.43	**3.40×**
Decoder	1,073.06	813.09	**1.32×**

2. Program and system improvements, compared within each timing scope

Workload	ROI: reference to final	Full process: after DSE to final	Initial to final End-to-end
MLP	11.641M → 41.593K (**280× faster**)	47.434M → 6.063M (**7.82× faster**)	**26.59×**
Decoder	435.377M → 6.029M (**72.2× faster**)	813.091M → 11.836M (**68.7× faster**)	**90.66×**

Final systems: MLP 1 CPU core + 4 GPUs × 1 CU; decoder 1 CPU core + 2 GPUs × 16 CUs.

MLP: 293 mm² / 287 W / 72.33°C. Decoder: 229 mm² / 251 W / 71.12°C.

Both workloads show gains under ROI and full-process timing.

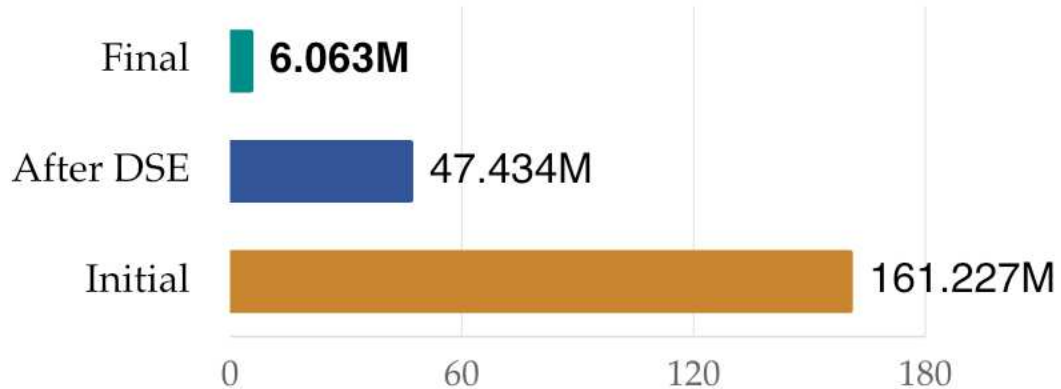
Decoder methods, hardware comparisons and validation evidence follow in the Q&A backup.

Conclusion

Chiplet DSE and workload restructuring improve both workloads.

Full-process latency (M cycles)

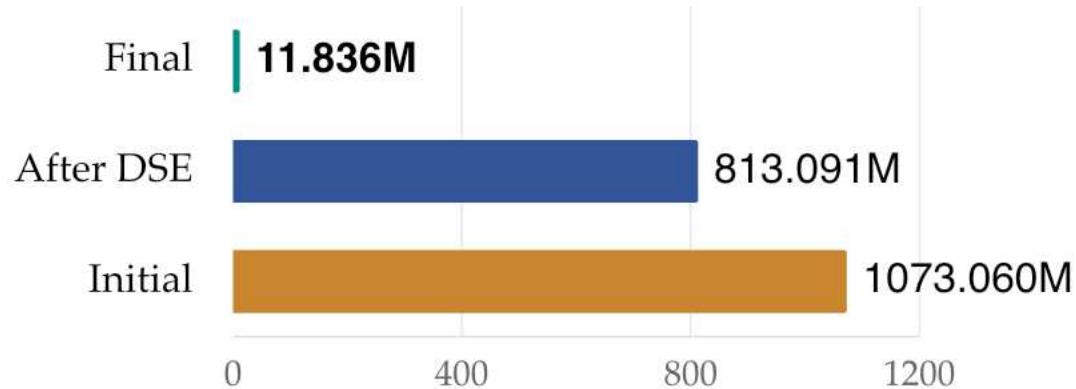
MLP



26.59× end-to-end

ROI improvement: 280×

Decoder



90.66× end-to-end

ROI improvement: 72.2×

Thank you

Questions and discussion

Q&A Guide



A **Why use an agent for cross-layer DSE?** 25–26

25 Cost and diagnosis 26 Agent and BO/GA

B **How are configurations proposed and evaluated?** 27–28

27 Proposal 28 Evaluation and records

C **Why these MLP resources and links?** 29–32

29 CU use 30 Placement 31 Link latency 32 −O3

D **Where do the gains come from?** 33–35

33 Optimization steps 34 Timing scopes 35 Claim limits

E **How is the decoder optimized?** 36–40

36 Shards 37 GPU shape 38 Network 39 Replies 40 Gains

F **How are results validated?** 41–44

41 Repeats 42 Repairs index 43 Physical checks 44 Decisions

G **What did the simulator repairs change?** 45–46

45 Evaluator repairs 46 ROI accounting probe

Expensive Evaluations Need Diagnosis



Wall-clock per converged stock run

Workload	Median	Maximum
MLP (n=46)	0.45 h	4.76 h
Decoder (n=8)	9.59 h	15.49 h

A tested change can be slower

MLP ROI: −O2 43,627 cycles
−O3 47,883 cycles (9.8% slower)

We kept −O2 after inspecting the execution timeline.

Agent Support Beyond Parameter Search



Configuration selection

BO or GA can propose candidates using the same evaluator and physical constraints.

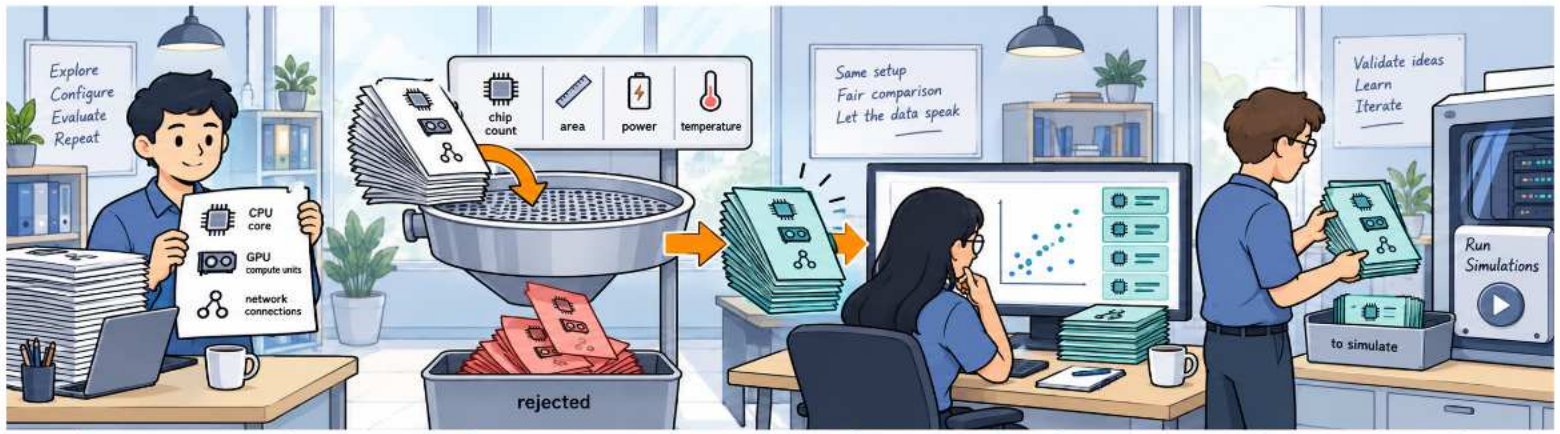
Agent-assisted execution changes

Read logs and source code.
Revise kernels and data placement.
Review CPU/GPU coordination.

Our workflow combines configuration search with workload restructuring.

Configuration Flow: Candidate Proposal

The proposal driver screens configurations and selects candidates for detailed evaluation.

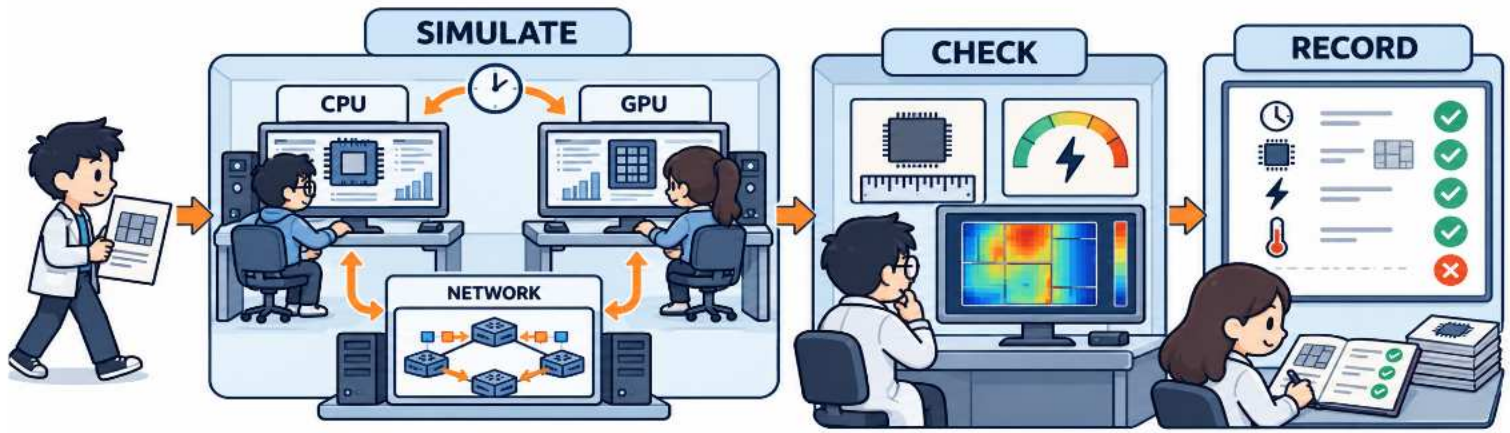


Configuration inputs
CPU cores, GPU CUs, topology,
flit size, virtual channels and buffers.

Screen and propose
Check physical limits, rank with a prior,
then write a proposal queue for review.

Configuration Flow: Evaluation and Records

Detailed timing and physical checks produce a recorded result for each evaluated configuration.

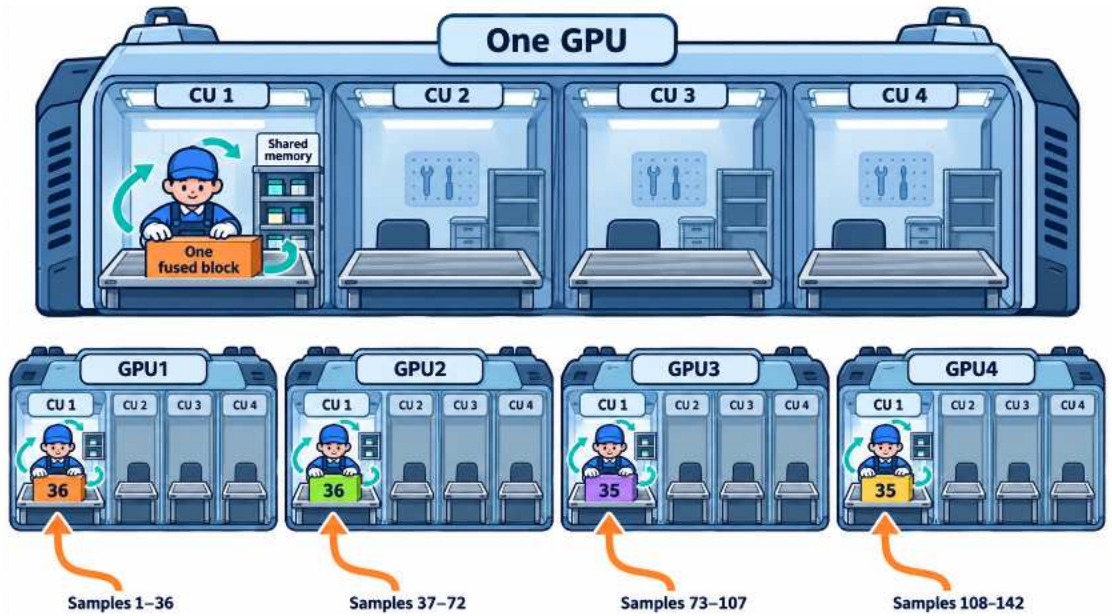


Timing evaluation
CPU/GPU simulation exchanges messages.
PopNet delays feed the next timing round.

Physical checks and output
Evaluate area, power and HotSpot temperature.
Record runtime, constraints and evidence.

Fusion Makes One CU Enough for Each MLP Shard

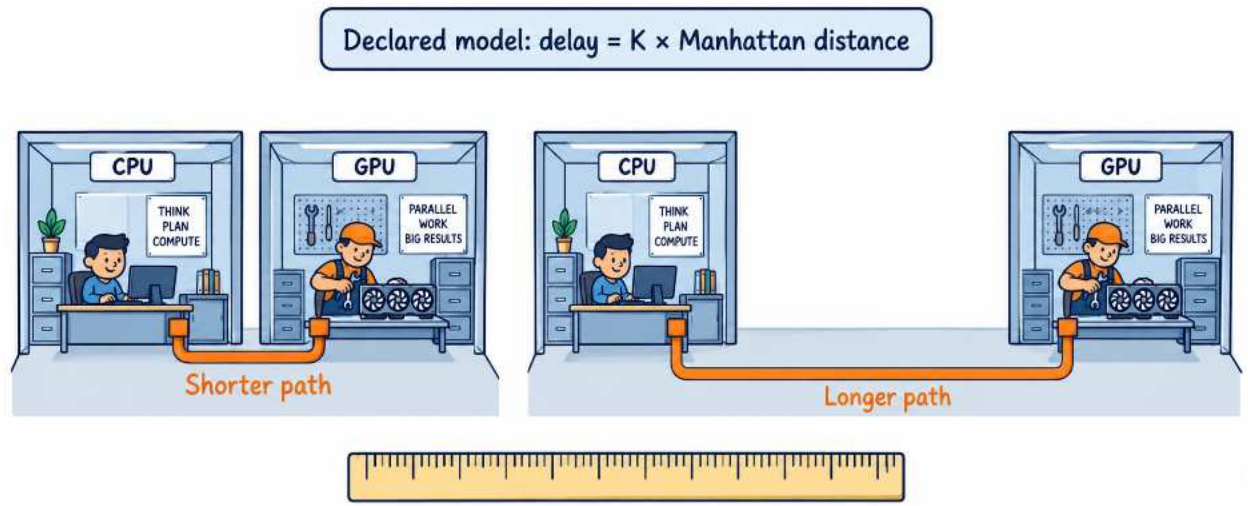
Take one GPU: its fused block runs on one CU and reuses shared memory.



Four GPUs process 36 / 36 / 35 / 35 samples concurrently; the fused path uses ≤48 KB shared memory.

Placement Connects Geometry to Link Delay

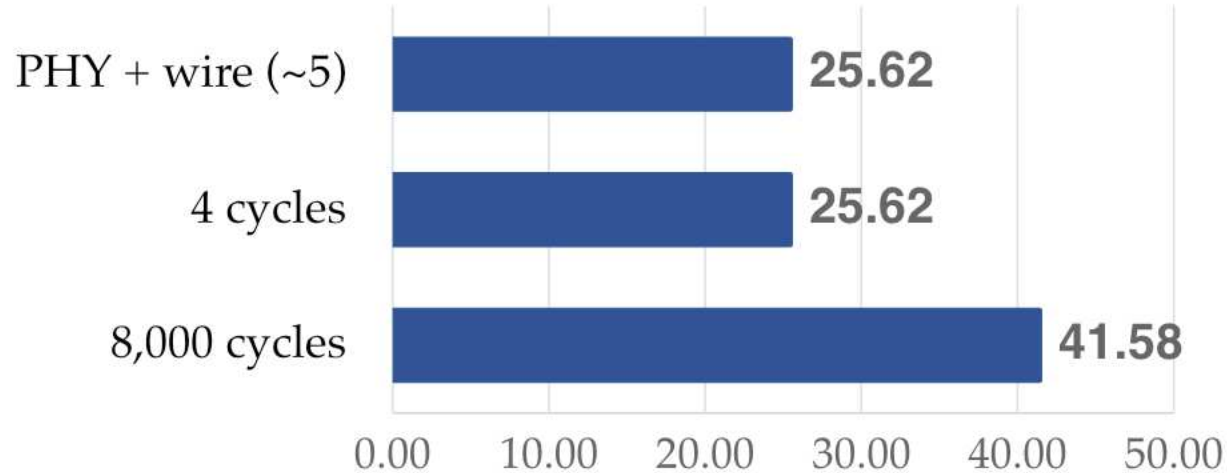
The declared model converts Manhattan distance into communication delay.



MLP reference: 1,000 cycles/mm × 8 mm = 8,000 cycles. K is a modeling parameter, not a silicon measurement.

Backup: Link Latency Changes the Final MLP Result

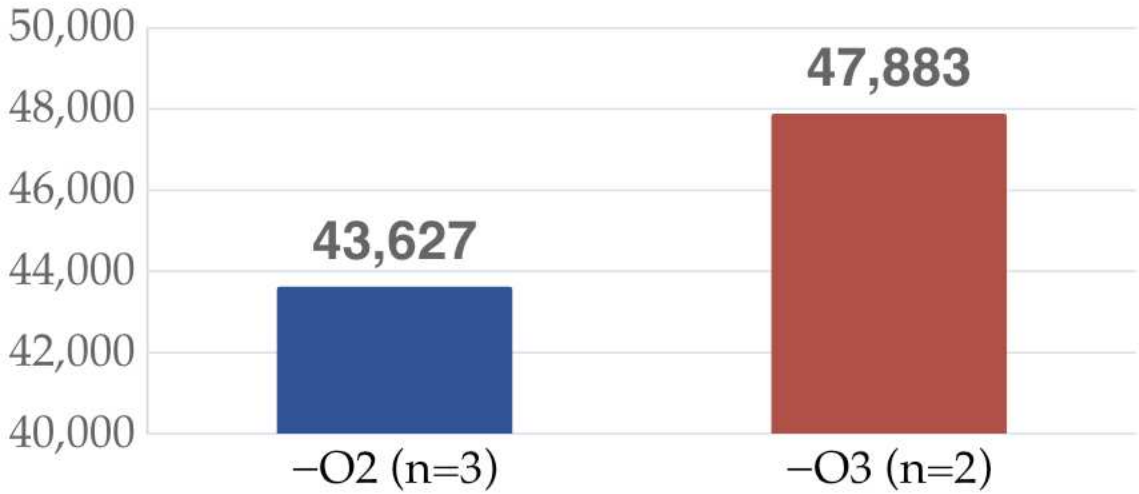
Runtime (k cycles), same r14 program



The link model must accompany placement results

Two runs per setting; the 4- and 5-cycle cases nearly coincide.

Backup: −O3 Loses in the Matched MLP Comparison

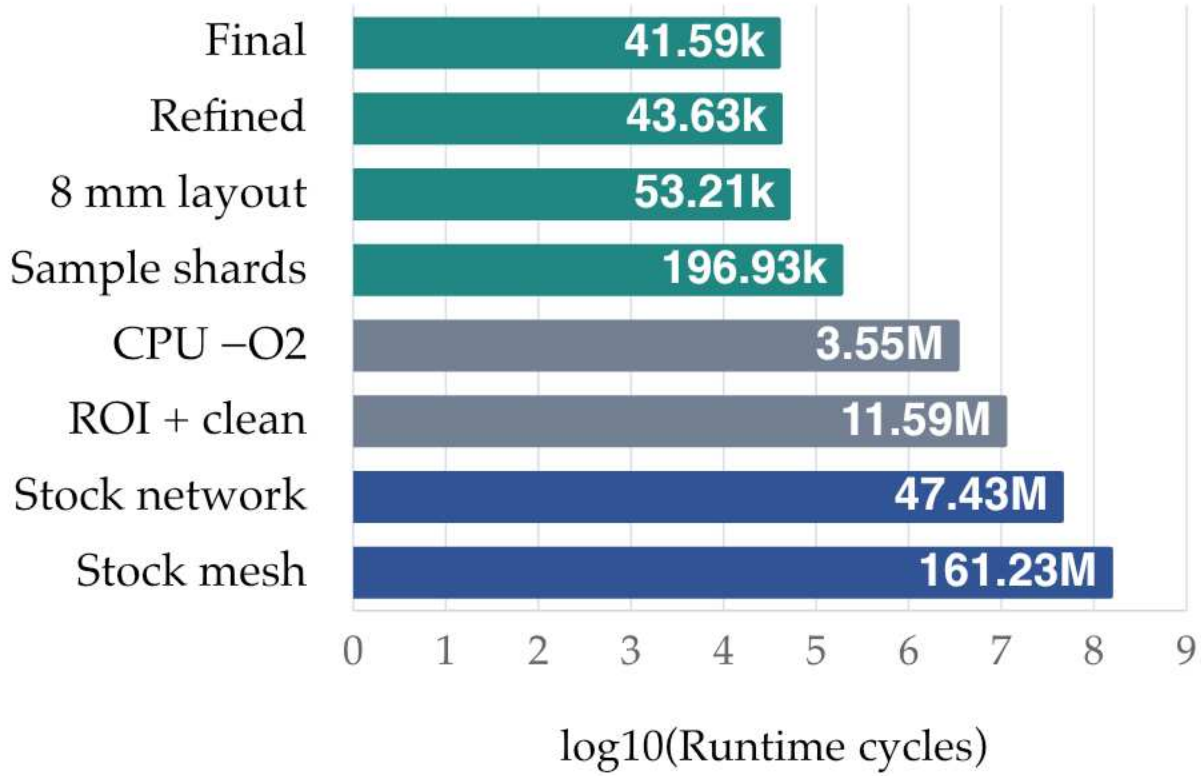


9.8%

Slower with −O3
Same r11 program and hardware

Zoomed axis: 40,000–50,000 cycles

Each MLP Step Removes a Different Source of Cost



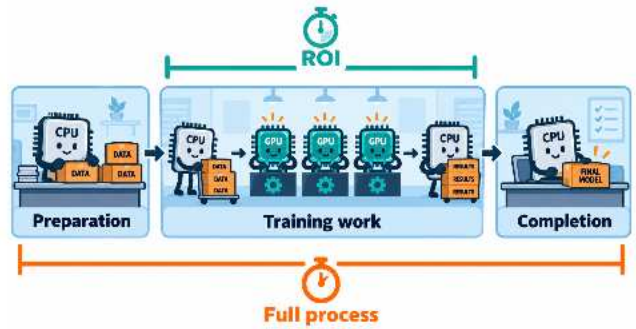
41,593

Final mean cycles
3 converged runs

Compound reduction Includes ROI, program and link-model changes

The Gains Remain Under Full-Process Timing

ROI and full-process timing measure different windows of the same run.



Timing scope	Reference	Final	Improvement
Full process	34.17M	6.06M	**5.6×**
ROI	11.64M	41.6K	**280×**

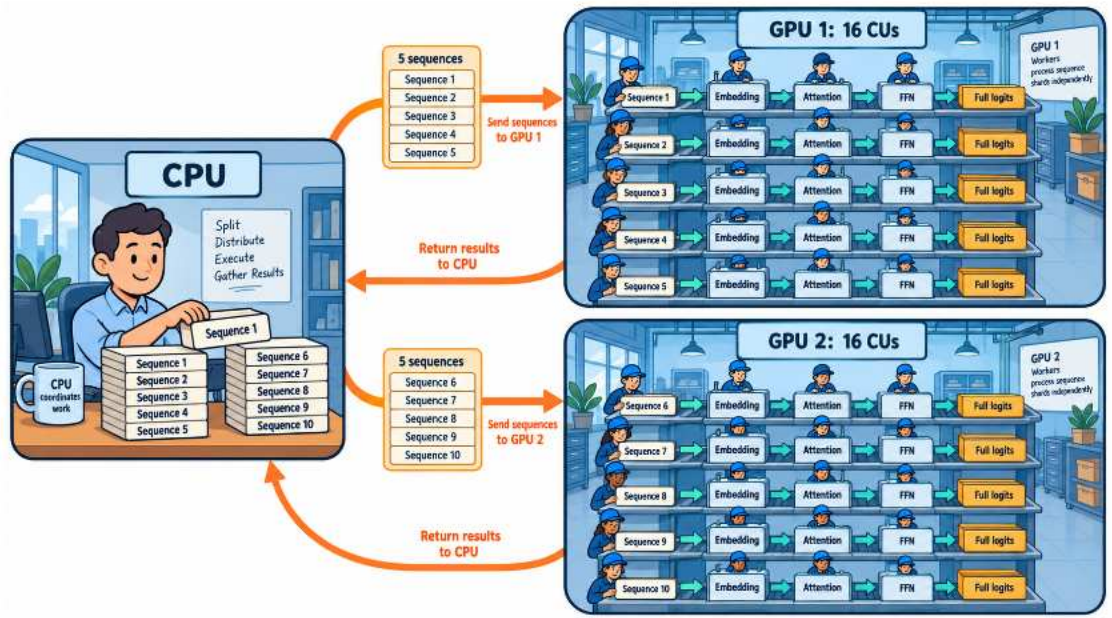
Early cleanup and compilation reduce preparation costs.
Later sharding and fusion mainly improve the ROI.

Backup: What Each Comparison Establishes

Claim	Supported interpretation
3.40× stock gain	Same stock program and process timing; configuration changes
Overall reduction	ROI, program, mapping and link-model changes combine
309 campaign records	Executed run rows, including repeats and failures
64 verification entries	55 PASS; 9 other statuses or qualifications
Cross-machine reproduction	Teammate-reported; original logs not locally supplied
Selected designs	Measured configurations under the declared models

Decoder Uses Larger Shards and Many GPU Blocks

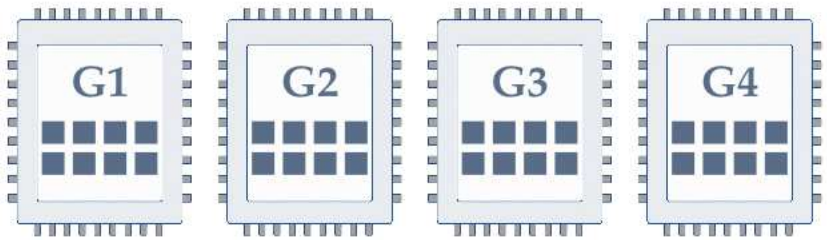
Split 10 complete sequences into two shards of 5 sequences.



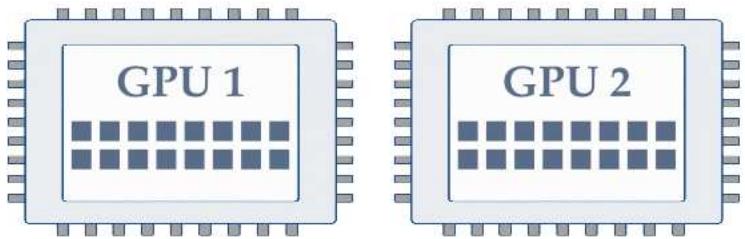
Each GPU uses many blocks to run embedding, attention and FFN, then returns full logits.

With 32 CUs, Two GPUs Beat Four GPUs

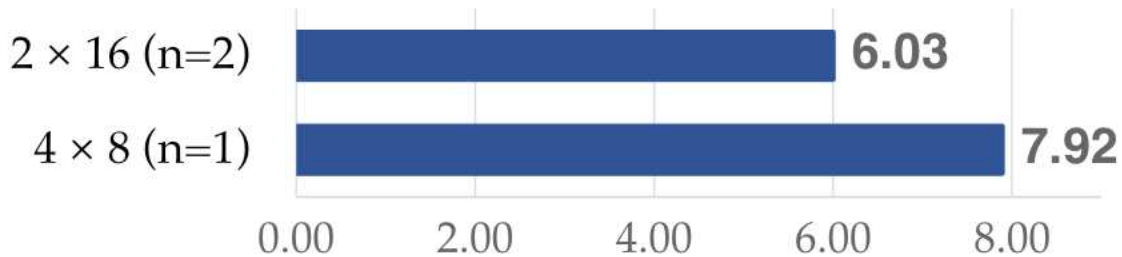
4 GPUs × 8 CU



2 GPUs × 16 CU



Runtime (M cycles)



23.9%

Lower Runtime
Same total CUs

System shape also changes shard size and placement.

Flit Width and Buffer Depth Improve Data Flow



Bytes per flit



Outstanding flits



Buffer	Flit 200	Flit 400
12	6.866M	6.441M
24	6.441M	6.225M

9.33%

Lower Runtime
200/12 to 400/24

Same r7 program, 2 × 16 CU, star network

A Smaller Reply Can Still Make the Program Slower



Argmax-only return

+133k

Extra GPU compute cycles

−24k

Communication cycles saved

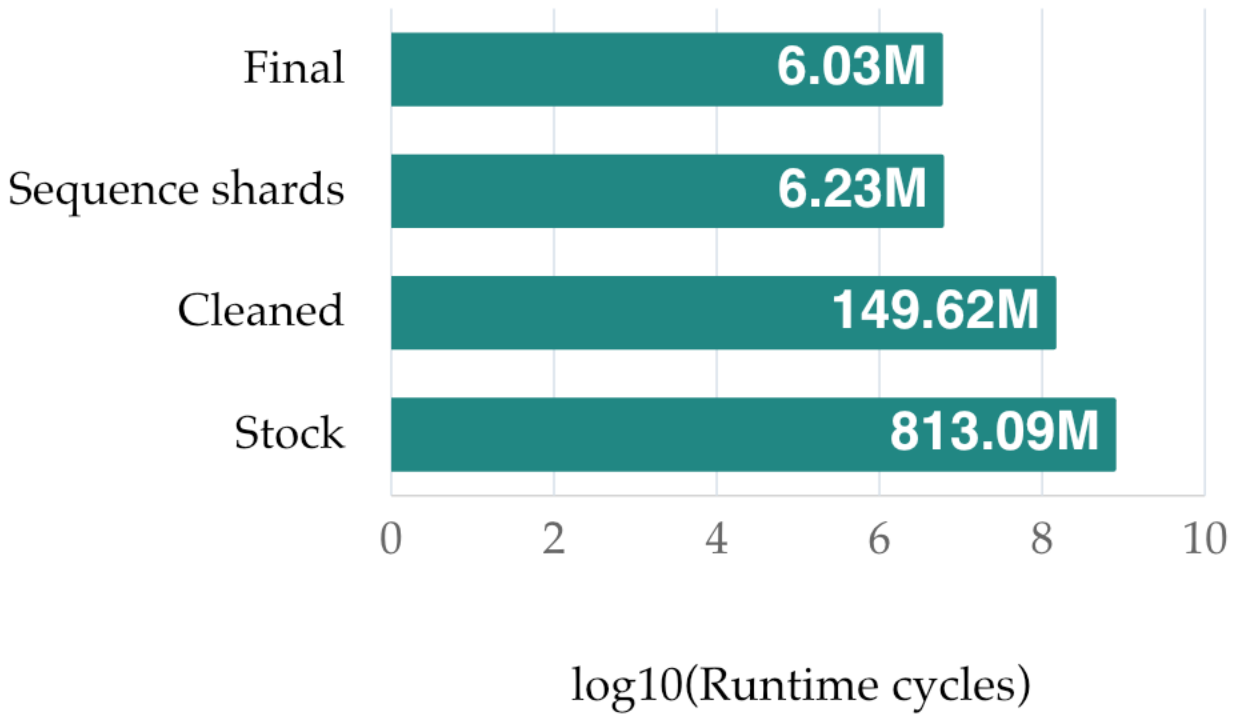
+1.82% Runtime

Measured comparison, n=2 each

Keep full logits

The added compute exceeds the transfer savings.

Backup: The Decoder Reduction Is Also Compound

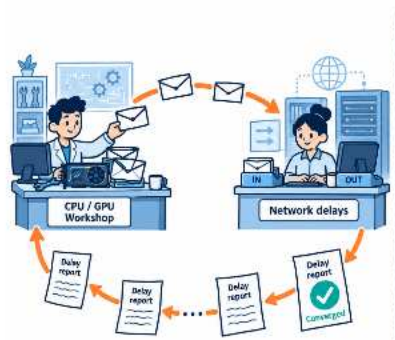


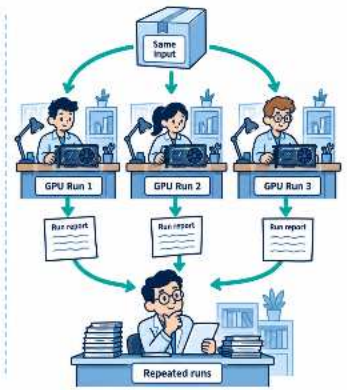
135×

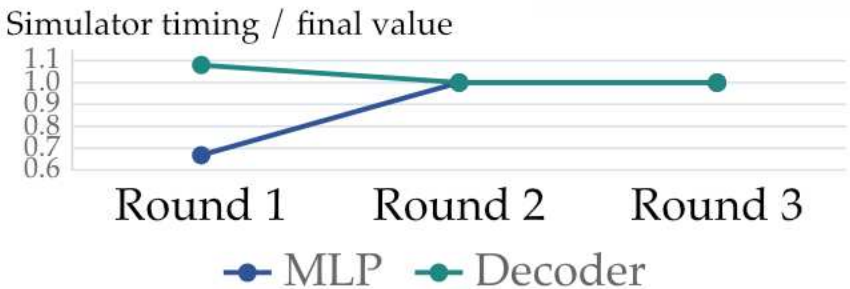
Compound reduction
Timing, program and links change

Timing Converges and Final Runs Repeat

Convergence checks one timing loop; repeats compare completed runs.







Frozen-input repeats

MLP: 41,580 / 41,595 / 41,605
Decoder: 6,029,082 twice

Final checksums match the validated computation.

Backup: Simulator Repairs Protect the Objective

Failure	Consequence	Evidence
Exit / handshake	Deadlock or race	Minimal reproductions
Thread bookkeeping	Wrong association	Source audit + checks
Latency matching	False convergence	Round + message audit
ROI residue	~100k stray cycles	Fixed-work ROI probe
Parser termination	Intermittent crash	Statistical evidence

Frozen v8 image includes the fixes and reproductions.

Backup: Physical Feasibility Uses a Frozen Model

Metric	MLP	Decoder	Limit
Chiplets	5	3	8
Area (mm²)	293	229	800
Power (W)	287	251	700
Temperature (°C)	72.33	71.12	80

HotSpot: 45°C ambient, r_convec = 0.01 K/W
64 × 64 grid maximum; 0.09 m spreader, 0.13 m sink

All nine final flow checks pass for each design.

Measured Decisions Remain Traceable

309

Recorded runs
282 OK, 27 failed

15

Human decision gates

9

Measured proposals rejected

Hypothesis	Measured decision
More CPU optimization	Reject −O3: 9.8% slower
Smaller decoder replies	Reject argmax: 1.82% slower
Change system shape	Select 2 × 16 CU for decoder

Evaluator Repairs Before Optimization

We repaired the evaluation stack before trusting configuration comparisons.



Run and finish

Compatibility fixes let programs start. Clock advancement lets the decoder finish.

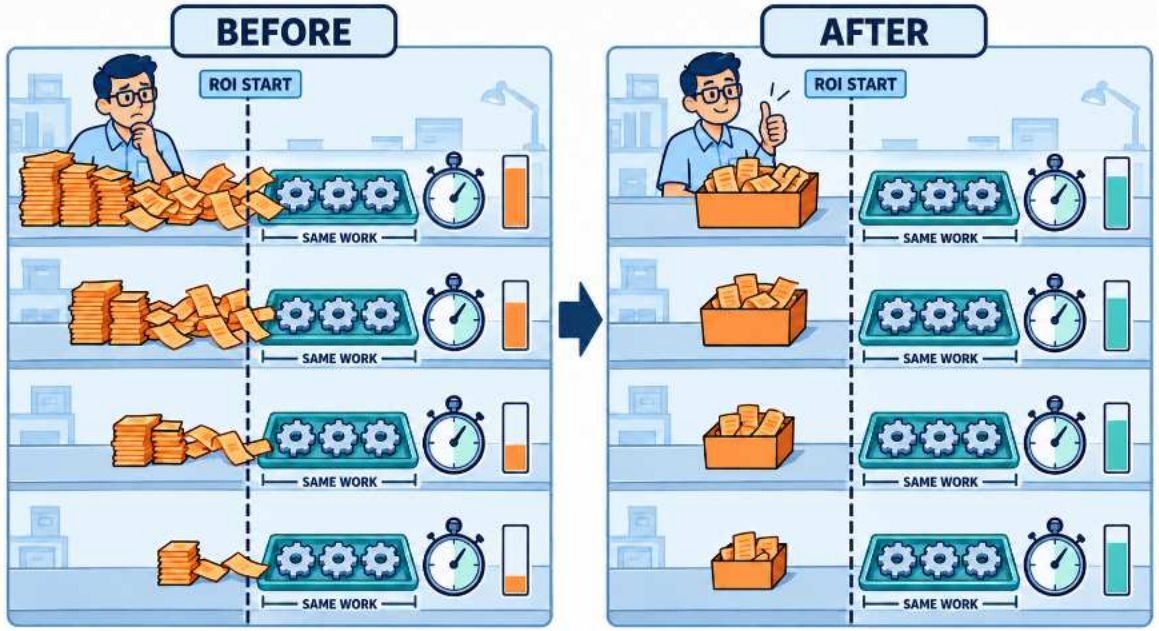
Measure and verify

Delay matching restores network costs. Controlled tests check the result.

Micro-test: 8,019 wrong cycles vs 202,087 reference cycles.

ROI Timing With the Same Computation

The probe changes preparation before the ROI and keeps the computation inside it fixed.



Same computation

Change only preparation before the ROI.

Before the repair

9,044–97,622 cycles

After the repair

6,118 cycles in all cases

Final MLP ROI: 41,593 cycles, using the corrected image v8.